

CURRYING IN JAVASCRIPT – A POWERFUL FUNCTIONAL PROGRAMMING CONCEPT



Lakshmiprasanna Vakada

Currying is an advanced JavaScript technique used in functional programming to break down functions into smaller, reusable functions. It improves code reusability, readability, and modularity.

Today, we will explore:

- ✓ What is Currying?
- ✓ How Does Currying Work?
- ✓ Simplifying with Arrow functions
- ✓ Examples of Currying in JavaScript
 - ✓ Real-World Use Cases

I. What is Currying?

Currying is a technique where a function that takes multiple arguments is transformed into a series of functions that take one argument at a time.

✓ Regular Function (Non-Curried)

```
function add(a, b) {  
  return a + b;  
}  
console.log(add(2, 3)); // Output: 5
```

- ◆ Here, `add(2, 3)` takes two arguments at once.

✓ Curried Function

```
function add(a) {  
  return function (b) {  
    return a + b;  
  };  
}  
console.log(add(2)(3)); // Output: 5
```

- ◆ Instead of passing all arguments at once, the function is broken into multiple functions that each take a single argument.

2. How Does Currying Work?

Currying allows us to partially apply functions.

✓ Example: Converting a Normal Function to a Curried Function

```
// Normal function
function multiply(a, b, c) {
  return a * b * c;
}
console.log(multiply(2, 3, 4)); // Output: 24

// Curried function
function curriedMultiply(a) {
  return function (b) {
    return function (c) {
      return a * b * c;
    };
  };
}
console.log(curriedMultiply(2)(3)(4)); // Output: 24
```

- ◆ The curried function takes one argument at a time and returns another function until all arguments are provided.

3. Simplifying with Arrow Functions

We can write a curried function using arrow functions for cleaner syntax:

```
const curriedMultiply = a => b => c =>  
  a * b * c;  
  
console.log(curriedMultiply(2)(3)(4));  
// Output: 24
```

- ◆ This is a more compact way to define curried functions.

4. Why Use Currying?

✓ Reusability & Function Composition

```
const multiplyBy2 = curriedMultiply(2);  
// Partially apply the function
```

```
console.log(multiplyBy2(3)(4)); //
```

Output: 24

```
console.log(multiplyBy2(5)(6)); //
```

Output: 60

- ◆ The function `multiplyBy2` is pre-configured with `a = 2`, making it reusable for different values of `b` and `c`.

5. Real-World Use Cases of Currying

✓ 1. Event Handlers

Currying is useful for handling dynamic event listeners:

```
const handleEvent = eventType =>
  element => callback => {
    element.addEventListener(eventType,
      callback);
  };

// Usage
const clickHandler =
  handleEvent("click");
clickHandler(document.getElementById("
  btn"))(() => console.log("Button
  clicked!"));
```


✓ 2. API Calls (Pre-Filled Configuration)

```
const fetchData = baseUrl => endpoint
=>
fetch(`${baseUrl}${endpoint}`).then(res
=> res.json());
// Pre-configure API with base URL
const getGithubData =
fetchData("https://api.github.com");

getGithubData("/users/octocat").then(da
ta => console.log(data)); // Fetches
user data
```

- ◆ This lets us call APIs without repeating the base URL.